Learning a Neural Network Controller for Ship Heading Control

Technical report (PID demonstrations -> neural network imitation) | 2026-02-27

1. Introduction

Ship heading control is the task of steering a vessel so that its yaw angle (heading) tracks a desired command. This project implements heading control in simulation and compares two controllers: (i) a hand-designed PID-based demonstrator used to generate training data, and (ii) a learned neural network (NN) controller trained to imitate the demonstrator from state-action examples.

Workflow: (1) run a closed-loop simulation using the PID controller, starting from (500, 400) and a random initial heading; (2) log demonstration state-action pairs; (3) train an NN that maps the current ship state and heading error to a rudder command; and (4) evaluate the NN in closed loop on multiple (initial heading, desired heading) test cases.

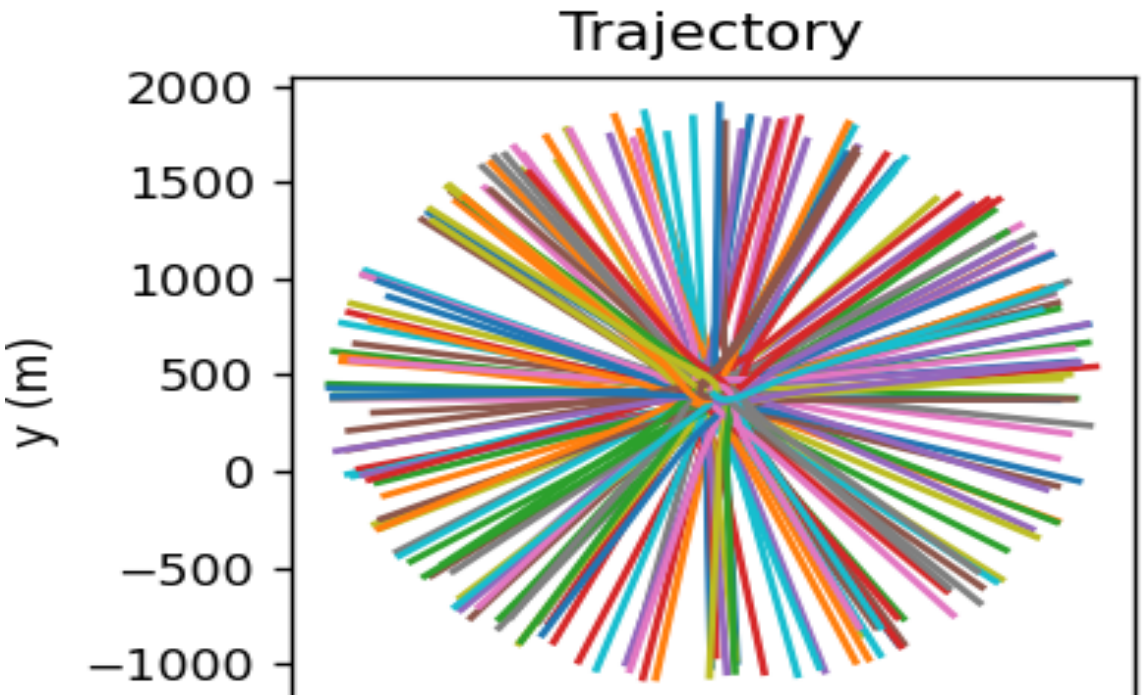

Figure 1: Overview of demonstration trajectories. Each line is a PID-controlled run starting at (500, 400) with random initial and target headings; after turning, the ship proceeds approximately straight at the achieved heading.

2. PID Demonstrator and Data Collection

The demonstrator computes rudder commands from the wrapped heading error $e_\psi = \mathrm{wrap}(\psi_{des} - \psi)$ (wrapped to $[-\pi, \pi]$). The controller uses fixed propulsion (commanded RPM = 80) and computes a rudder command $u_\delta = k_p e_\psi + k_i \int e_\psi \, dt + k_d \, de_\psi/dt$. In the provided implementation, $k_i = 0$ so the behavior is effectively PD.

Parameter	Value
k_p (heading)	0.3
k_i (heading)	0.0
k_d (heading)	3.5
Integral clamp	±20 deg (in radians internally)
Commanded RPM	80.0
Actuator rudder limit	±45 deg
Actuator rudder rate	3.7 deg/s

Demonstrations are generated by running 200 trajectories with timestep dt = 1 s and duration T = 200 s. Each trajectory starts at (x, y) = (500, 400) with a uniformly random initial heading $\psi(0) \in [0, 2\pi)$, and tracks a uniformly random target heading $\psi_{des} \in [0, 2\pi)$. The simulator logs the ship state (x, y, ψ, u, v, r), the commanded inputs (cmd_rpm, cmd_rudder), and actuator outputs (act_rpm, act_rudder). For training, the dataset also stores the scalar heading error e_ψ and the target heading.

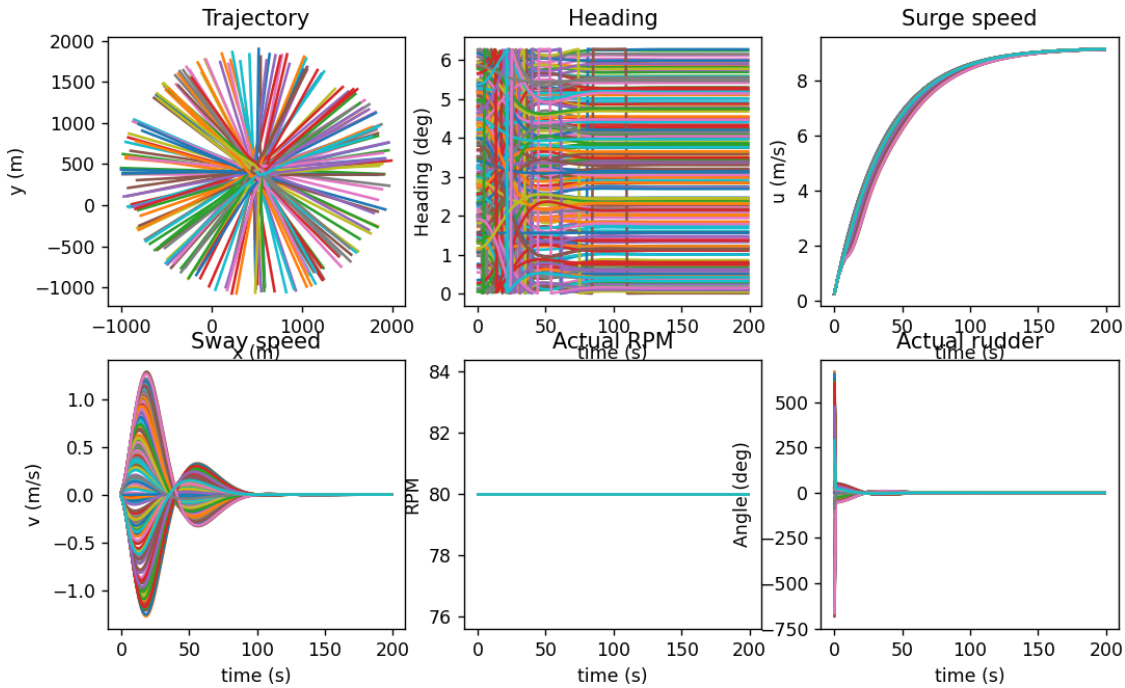

Figure 2: Example visualization of the generated demonstration data (overlaid trajectories and signals). The heading plot shows wrapped angles; surge speed u converges due to fixed propulsion; sway speed v is primarily transient during turning.

3. Neural Network Controller and Closed-Loop Evaluation

The NN controller is trained by behavioral cloning: it learns to predict the demonstrator's rudder command from the current state. To support arbitrary desired headings, the network input includes the wrapped heading error e_ψ, rather than the absolute target angle. In closed loop, the controller recomputes $e_\psi = \mathrm{wrap}(\psi_{des} - \psi)$ online and feeds it to the NN.

Item	Specification
Input features (4)	[e_psi, u, v, r] where e_psi in radians; u,v in m/s; r in rad/s
Output (1)	cmd_rudder (deg)
Network	MLP: Linear(4,32)-ReLU-Linear(32,32)-ReLU-Linear(32,1)
Hidden layers	2
Hidden units	32 per layer
Loss	Mean squared error (MSE)
Optimizer	Adam, learning rate 5e-3
Batch size	512
Epochs	100

Closed-loop evaluation runs the simulator with the NN controller (same dt and T as the demonstrations). Figure 3 shows six test cases with different initial and desired headings. The trajectories show an initial turning transient followed by near-straight motion, indicating convergence to (and maintenance of) the commanded heading. In a representative benchmark run, final heading errors were below 1 degree across all six cases.

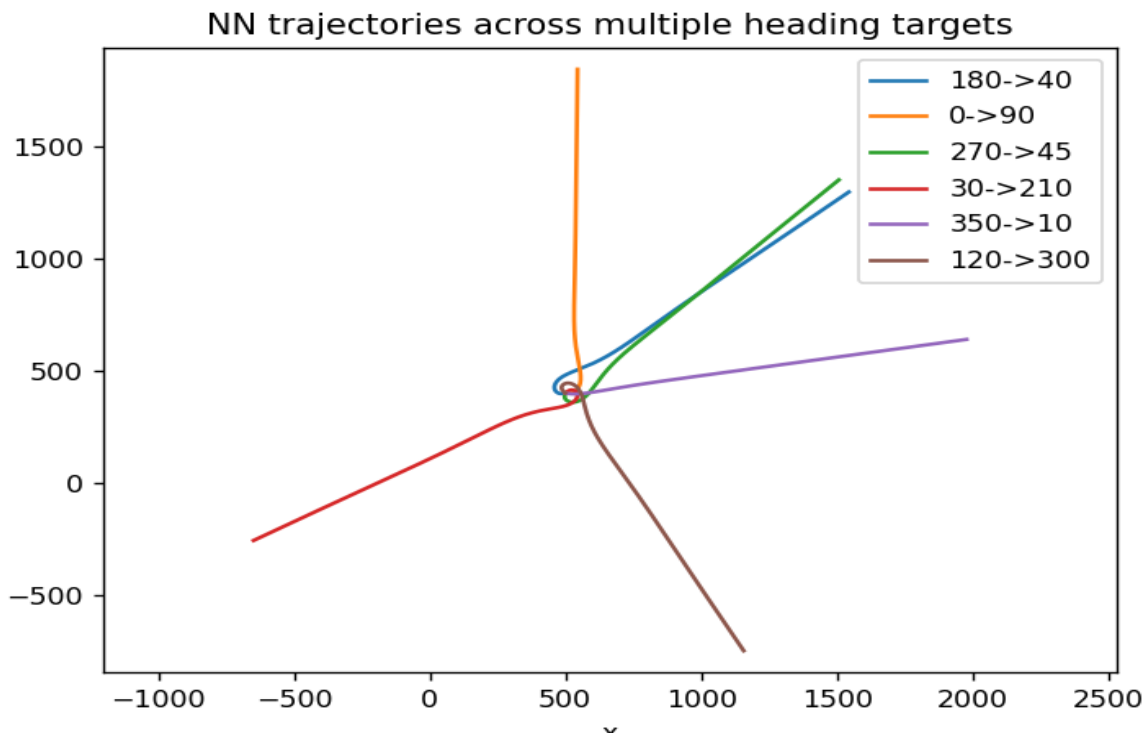

Figure 3: NN controller in closed loop on six heading commands (init->target, degrees). Curved segments correspond to turning; straight segments indicate heading convergence.

Conclusion. A PID demonstrator generated heading-control demonstrations in simulation. A compact MLP controller trained on (e_psi, u, v, r) reproduces the demonstrator's steering behavior and tracks arbitrary desired heading commands in closed-loop simulation.